

LAPORAN PRAKTIKUM

STRUKTUR DATA

ALGORITMA SORTING



Disusun Oleh:

Gina Ramadhani

Nim: 2511533014

Dosen Pengampu: DR. Wahyudi, S.T, M.T

Asisten Praktikum: Sherly Sukmadira

DAPERTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum ini dengan baik. Laporan ini berisi pembahasan mengenai beberapa algoritma sorting yang telah dipelajari dan diimplementasikan menggunakan bahasa pemrograman Java, yaitu Shell Sort, Quick Sort, Merge Sort, serta penerapan sorting pada studi kasus playlist lagu. Melalui praktikum ini, penulis dapat memahami konsep pengurutan data serta cara kerja masing-masing algoritma dalam menyelesaikan permasalahan pengolahan data.

Penulis menyadari bahwa laporan ini masih memiliki keterbatasan dan kekurangan, baik dari segi penyajian maupun pembahasan materi. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan laporan di masa mendatang. Semoga laporan ini dapat memberikan manfaat serta menambah wawasan mengenai penerapan algoritma sorting dalam pemrograman Java.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori	2
2.2 Praktikum Class “ShellSort_2511533014”	3
2.3 Praktikum Class “QuickSort_2511533014”	5
2.4 Praktikum Class “MergeSort_2511533014”	10
2.5 Praktikum Class “BubleSortGUI”	15
2.6 Laporan Tugas Praktikum	24
BAB III PENUTUP	28
3.1 Kesimpulan.....	28
DAFTAR PUSTAKA.....	29

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data (sorting) merupakan salah satu proses penting dalam pemrograman yang digunakan untuk menyusun data agar lebih teratur dan mudah diolah. Terdapat berbagai algoritma sorting yang memiliki cara kerja dan tingkat efisiensi yang berbeda, seperti Shell Sort, Quick Sort, dan Merge Sort. Oleh karena itu, praktikum ini dilakukan untuk mempelajari konsep, implementasi, serta perbandingan cara kerja beberapa algoritma sorting menggunakan bahasa pemrograman Java. Selain itu, algoritma tersebut diterapkan pada studi kasus pengurutan data playlist lagu untuk memahami penggunaannya dalam permasalahan nyata.

1.2 Tujuan Praktikum

1. Memahami konsep dasar algoritma sorting.
2. Mempelajari cara kerja algoritma Shell Sort, Quick Sort, dan Merge Sort.
3. Mengimplementasikan algoritma sorting menggunakan bahasa pemrograman Java.
4. Mengurutkan data berdasarkan kriteria tertentu pada studi kasus playlist lagu.
5. Menganalisis hasil pengurutan yang diperoleh dari setiap algoritma.

1.3 Manfaat Praktikum

1. Menambah pemahaman mengenai algoritma sorting dan penerapannya.
2. Meningkatkan kemampuan pemrograman Java dalam pengolahan data.
3. Memahami perbedaan cara kerja dan efisiensi berbagai algoritma sorting.
4. Melatih kemampuan analisis terhadap hasil pengurutan data.
5. Menjadi dasar dalam mempelajari struktur data dan algoritma yang lebih kompleks.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Sorting adalah proses mengurutkan data berdasarkan kriteria tertentu, seperti urutan naik (ascending) atau urutan turun (descending). Dalam pemrograman, sorting digunakan untuk mempermudah pencarian, pengelolaan, dan analisis data. Algoritma sorting memiliki berbagai metode dengan karakteristik dan tingkat efisiensi yang berbeda-beda sesuai kebutuhan pengolahan data.

Shell Sort merupakan pengembangan dari Insertion Sort yang bekerja dengan membandingkan elemen yang berjarak tertentu (gap) sehingga proses pengurutan menjadi lebih cepat. Quick Sort adalah algoritma sorting yang menggunakan metode divide and conquer dengan memilih sebuah pivot, kemudian membagi data menjadi dua bagian sebelum diurutkan secara rekursif. Merge Sort juga menggunakan metode divide and conquer, yaitu dengan membagi array menjadi bagian-bagian kecil, mengurutkannya, lalu menggabungkannya kembali hingga menjadi data yang terurut.

Pada praktikum ini, algoritma sorting diimplementasikan menggunakan bahasa pemrograman Java untuk mengurutkan data numerik maupun data playlist lagu. Melalui implementasi tersebut dapat dipahami bagaimana proses pengurutan berlangsung serta perbedaan cara kerja masing-masing algoritma dalam menghasilkan data yang terurut.

2.2 Praktikum Class “ShellSort_2511533014”

```
1 package pekan8_2511533014;
2
3 public class ShellSort_2511533014 {
4
5     public static void shellSort_3014(int[] A_3014) {
6         int n_3014 = A_3014.length;
7         int gap_3014 = n_3014/2;
8         while (gap_3014 > 0) {
9             for (int i_3014 = gap_3014; i_3014 < n_3014; i_3014++) {
10                 int temp_3014 = A_3014[i_3014];
11                 int j_3014 = i_3014;
12                 while (j_3014 >= gap_3014 && A_3014 [j_3014 - gap_3014] > temp_3014) {
13                     A_3014 [j_3014] = A_3014[j_3014 - gap_3014];
14                     j_3014 = j_3014 - gap_3014;
15                 }
16                 A_3014 [j_3014] = temp_3014;
17             }
18             gap_3014 = gap_3014 / 2;
19         }
20     }
21     public static void main (String[] args) {
22         int[] data_3014 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
23
24         System.out.print("Sebelum: ");
25         printArray(data_3014);
26
27         shellSort_3014(data_3014);
28
29         System.out.print("Sesudah (Shell Sort): ");
30         printArray(data_3014);
31     }
32
33     public static void printArray(int[] arr) {
34         for (int i_3014 : arr) System.out.print(i_3014 + " ");
35         System.out.println();
36     }
37 }
```

Gambar 2.1 Kode Program praktikum class “ShellSort”

Program ShellSort_2511533014 digunakan untuk mengurutkan data angka dalam array menggunakan algoritma Shell Sort. Shell Sort merupakan pengembangan dari Insertion Sort yang melakukan pengurutan dengan membandingkan elemen yang berjarak tertentu (*gap*) terlebih dahulu, kemudian jarak tersebut diperkecil hingga menjadi 1. Dengan cara ini, proses pengurutan menjadi lebih cepat dibandingkan Insertion Sort biasa untuk data yang cukup banyak.

Berikut Penjelasan langkah-langkah kerjanya:

1. `public static void shellSort_3014(int[] A_3014)`
→ Method utama yang berfungsi untuk mengurutkan array menggunakan algoritma Shell Sort.
2. `int n_3014 = A_3014.length;`
→ Menyimpan jumlah elemen array ke variabel `n_3014`.
3. `int gap_3014 = n_3014 / 2;`

- Menentukan jarak awal (*gap*) sebesar setengah dari panjang array.
- 4. `while (gap_3014 > 0)`
 - Perulangan yang akan terus berjalan selama nilai *gap* masih lebih besar dari 0.
- 5. `for (int i_3014 = gap_3014; i_3014 < n_3014; i_3014++)`
 - Menelusuri elemen array mulai dari indeks sesuai nilai *gap*.
- 6. `int temp_3014 = A_3014[i_3014];`
 - Menyimpan sementara nilai elemen yang akan diposisikan ke tempat yang benar.
- 7. `int j_3014 = i_3014;`
 - Variabel indeks untuk proses pergeseran elemen.
- 8. `while (j_3014 >= gap_3014 && A_3014[j_3014 - gap_3014] > temp_3014)`
 - Membandingkan elemen sebelumnya yang berjarak *gap*. Jika lebih besar dari *temp_3014*, maka elemen digeser ke kanan.
- 9. `A_3014[j_3014] = A_3014[j_3014 - gap_3014];`
 - Menggeser elemen yang lebih besar ke posisi berikutnya
- 10. `j_3014 = j_3014 - gap_3014;`
 - Memindahkan indeks ke posisi sebelumnya sesuai nilai *gap*.
- 11. `A_3014[j_3014] = temp_3014;`
 - Menempatkan nilai sementara (*temp_3014*) pada posisi yang tepat.
- 12. `gap_3014 = gap_3014 / 2;`
 - Mengurangi nilai *gap* menjadi setengahnya hingga akhirnya bernilai 1 lalu 0.
- 13. `int[] data_3014 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};`
 - Array yang akan diurutkan.
- 14. `printArray(data_3014);`
 - Menampilkan isi array sebelum dan sesudah proses pengurutan.
- 15. `shellSort_3014(data_3014);`
 - Memanggil method Shell Sort untuk mengurutkan data.
- 16. `public static void printArray(int[] arr)`
 - Method untuk mencetak seluruh elemen array ke layar.
- 17. `System.out.println();`

→ Membuat tampilan output berikutnya dimulai dari baris baru agar hasil lebih rapi.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.2 dibawah ini.

```
<terminated> ShellSort_2511533014 [Java Application] C:\Users
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

Gambar 2.2 Output kode program praktikum ShellSort

2.2.1 Analisis

Output di atas menunjukkan bahwa algoritma Shell Sort berhasil mengurutkan data dari kondisi awal 3 10 4 6 8 9 7 2 1 5 menjadi 1 2 3 4 5 6 7 8 9 10 secara menaik (*ascending*). Proses pengurutan dilakukan dengan membandingkan elemen yang memiliki jarak tertentu (*gap*), kemudian jarak tersebut diperkecil secara bertahap hingga menjadi 1. Dengan metode ini, elemen-elemen yang berada jauh dari posisi seharusnya dapat dipindahkan lebih cepat, sehingga proses pengurutan menjadi lebih efisien. Hasil akhir menunjukkan bahwa seluruh elemen array telah tersusun dari nilai terkecil hingga terbesar dengan benar.

2.3 Praktikum Class “QuickSort_2511533014”

```
1 package pekan8_2511533014;
2
3 public class QuickSort_2511533014 {
4     static void swap_3014(int[] arr_3014, int i_3014, int j_3014)
5     {
6         int temp_3014 = arr_3014[i_3014];
7         arr_3014[i_3014] = arr_3014[j_3014];
8         arr_3014[j_3014] = temp_3014;
9     }
10
11     // Metode tambahan untuk mengatur pivot menggunakan Median-of-Three
12     static void medianOfThree_3014(int[] arr_3014, int low_3014, int high_3014)
13     {
14         int mid_3014 = low_3014 + (high_3014 - low_3014) / 2;
15         // Urutkan elemen low, mid, dan high
16         if (arr_3014[low_3014] > arr_3014[mid_3014]) {
17             swap_3014(arr_3014, low_3014, mid_3014);
18         }
19         if (arr_3014[low_3014] > arr_3014[high_3014]) {
20             swap_3014(arr_3014, low_3014, high_3014);
21         }
22         if (arr_3014[mid_3014] > arr_3014[high_3014]) {
23             swap_3014(arr_3014, mid_3014, high_3014);
24         }
25         swap_3014(arr_3014, mid_3014, high_3014);
26
27 }
```



```

28 static int partition_3014(int[] arr_3014, int low_3014, int high_3014)
29 {
30     // Panggil fungsi medianOfThree sebelum menentukan pivot
31     medianOfThree_3014(arr_3014, low_3014, high_3014);
32     int pivot_3014 = arr_3014[high_3014]; // Sekarang arr[high] sudah berisi nilai median
33     int i_3014 = (low_3014 - 1);
34     for (int j_3014 = low_3014; j_3014 <= high_3014 - 1; j_3014++) {
35         // Jika elemen saat ini lebih kecil dari atau sama dengan pivot
36         if (arr_3014[j_3014] < pivot_3014) {
37             // Increment indeks elemen yang lebih kecil
38             i_3014++;
39             swap_3014(arr_3014, i_3014, j_3014);
40         }
41     }
42     swap_3014(arr_3014, i_3014 + 1, high_3014);
43     return (i_3014 + 1);
44 }
45
46 static void quickSort_3014(int[] arr_3014, int low_3014, int high_3014)
47 {
48     if (low_3014 < high_3014) {
49         int pi_3014 = partition_3014(arr_3014, low_3014, high_3014);
50         quickSort_3014(arr_3014, low_3014, pi_3014 - 1);
51         quickSort_3014(arr_3014, pi_3014 + 1, high_3014);
52     }
53 }
54
55 public static void printArr_3014(int[] arr_3014)
56 {
57     for (int i_3014 = 0; i_3014 < arr_3014.length; i_3014++) {
58         System.out.print(arr_3014[i_3014] + " ");
59     }
60     System.out.println();
61 }
62
63 public static void main(String[] args)
64 {
65     int[] arr_3014 = {10, 7, 8, 9, 1, 5};
66     int N_3014 = arr_3014.length;
67
68     System.out.print("Data sebelum diurutkan: ");
69     printArr_3014(arr_3014);
70
71     quickSort_3014(arr_3014, 0, N_3014 - 1);
72
73     System.out.print("Data Terurut quicksort: ");
74     printArr_3014(arr_3014);
75 }
76 }

```

Gambar 2.3 Kode program praktikum class “QuickSort”

Program QuickSort_2511533014 digunakan untuk mengurutkan data dalam array dengan algoritma Quick Sort. Method swap_3014() menukar elemen, medianOfThree_3014() memilih pivot yang optimal, partition_3014() membagi data berdasarkan pivot, dan quickSort_3014() mengurutkan data secara rekursif. Indeks penting yang digunakan yaitu low_3014 (awal array), high_3014 (akhir array), i_3014 dan j_3014 (proses perbandingan data), serta pi_3014 (posisi pivot).

Berikut penjelasan langkah-langkah kerjanya:

1. static void swap_3014(int[] arr_3014, int i_3014, int j_3014)

→ Method untuk menukar posisi dua elemen dalam array.

2. `int temp_3014 = arr_3014[i_3014];`
→ Menyimpan sementara nilai elemen pada indeks `i_3014`.
3. `arr_3014[i_3014] = arr_3014[j_3014];`
→ Mengganti nilai indeks `i_3014` dengan nilai indeks `j_3014`.
4. `arr_3014[j_3014] = temp_3014;`
→ Menempatkan nilai sementara ke indeks `j_3014`.
5. `static void medianOfThree_3014(int[] arr_3014, int low_3014, int high_3014)`
→ Method untuk memilih pivot menggunakan teknik Median of Three.
6. `int mid_3014 = low_3014 + (high_3014 - low_3014) / 2;`
→ Menentukan indeks tengah array.
7. `if (arr_3014[low_3014] > arr_3014[mid_3014])`
→ Membandingkan elemen awal dan tengah.
8. `if (arr_3014[low_3014] > arr_3014[high_3014])`
→ Membandingkan elemen awal dan akhir.
9. `if (arr_3014[mid_3014] > arr_3014[high_3014])`
→ Membandingkan elemen tengah dan akhir.
10. `swap_3014(arr_3014, mid_3014, high_3014);`
→ Memindahkan nilai median ke posisi akhir sebagai pivot.
11. `static int partition_3014(int[] arr_3014, int low_3014, int high_3014)`
→ Method untuk membagi array berdasarkan pivot.
12. `medianOfThree_3014(arr_3014, low_3014, high_3014);`
→ Memanggil metode Median of Three untuk menentukan pivot.
13. `int pivot_3014 = arr_3014[high_3014];`
→ Menyimpan nilai pivot dari elemen terakhir.

14. `int i_3014 = (low_3014 - 1);`
→ Menyiapkan indeks untuk elemen yang lebih kecil dari pivot.
15. `for (int j_3014 = low_3014; j_3014 <= high_3014 - 1; j_3014++)`
→ Melakukan perulangan untuk membandingkan setiap elemen dengan pivot.
16. `if (arr_3014[j_3014] < pivot_3014)`
→ Mengecek apakah elemen lebih kecil dari pivot.
17. `i_3014++;`
→ Menggeser indeks elemen yang lebih kecil.
18. `swap_3014(arr_3014, i_3014, j_3014);`
→ Menukar elemen agar elemen kecil berada di sebelah kiri pivot.
19. `swap_3014(arr_3014, i_3014 + 1, high_3014);`
→ Menempatkan pivot pada posisi yang benar.
20. `return (i_3014 + 1);`
→ Mengembalikan posisi pivot setelah partition.
21. `static void quickSort_3014(int[] arr_3014, int low_3014, int high_3014)`
→ Method utama algoritma Quick Sort.
22. `if (low_3014 < high_3014)`
→ Memastikan masih ada data yang perlu diurutkan.
23. `int pi_3014 = partition_3014(arr_3014, low_3014, high_3014);`
→ Menentukan posisi pivot setelah proses partition.
24. `quickSort_3014(arr_3014, low_3014, pi_3014 - 1);`
→ Mengurutkan bagian kiri pivot secara rekursif.
25. `quickSort_3014(arr_3014, pi_3014 + 1, high_3014);`
→ Mengurutkan bagian kanan pivot secara rekursif.

26. `public static void printArr_3014(int[] arr_3014)`

→ Method untuk menampilkan isi array.

27. `for (int i_3014 = 0; i_3014 < arr_3014.length; i_3014++)`

→ Mengakses seluruh elemen array.

28. `System.out.print(arr_3014[i_3014] + " ");`

→ Menampilkan setiap elemen array.

29. `public static void main(String[] args)`

→ Method utama yang pertama kali dijalankan.

30. `int[] arr_3014 = {10, 7, 8, 9, 1, 5};`

→ Membuat data yang akan diurutkan.

31. `int N_3014 = arr_3014.length;`

→ Menyimpan jumlah elemen array.

32. `System.out.print("Data sebelum diurutkan: ");`

→ Menampilkan teks data sebelum diurutkan.

33. `printArr_3014(arr_3014);`

→ Menampilkan isi array sebelum proses sorting.

34. `quickSort_3014(arr_3014, 0, N_3014 - 1);`

→ Memanggil method Quick Sort untuk mengurutkan seluruh data.

35. `System.out.print("Data Terurut quicksort: ");`

→ Menampilkan teks hasil pengurutan.

36. `printArr_3014(arr_3014);`

→ Menampilkan array setelah berhasil diurutkan.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.4 dibawah ini.

```
<terminated> QuickSort_2511533014 [Java Application] C:\
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```

Gambar 2.4 Output kode program class “QuickSort”

2.3.1 Analisis

Output menunjukkan bahwa sebelum proses Quick Sort, data dalam array masih acak yaitu 10, 7, 8, 9, 1, 5. Setelah method quickSort_3014() dijalankan, data berhasil diurutkan secara ascending (dari kecil ke besar) menjadi 1, 5, 7, 8, 9, 10. Hal ini membuktikan bahwa algoritma Quick Sort dengan metode Median of Three sebagai pemilihan pivot bekerja dengan baik dalam membagi dan mengurutkan data hingga seluruh elemen berada pada urutan yang benar.

2.4 Praktikum Class “MergeSort_2511533014”

```
1 package pekan8_2511533014;
2
3 public class MergeSort_2511533014 {
4     void merge_3014(int arr_3014[], int l_3014, int m_3014, int r_3014)
5     {
6         // Find sizes of two subarrays to be merged
7         int n1_3014 = m_3014 - l_3014 + 1;
8         int n2_3014 = r_3014 - m_3014;
9         /* Create temp arrays */
10        int L_3014[] = new int[n1_3014];
11        int R_3014[] = new int[n2_3014];
12        /* Copy data to temp arrays */
13        for (int i_3014 = 0; i_3014 < n1_3014; ++i_3014)
14            L_3014[i_3014] = arr_3014[l_3014 + i_3014];
15        for (int j_3014 = 0; j_3014 < n2_3014; ++j_3014)
16            R_3014[j_3014] = arr_3014[m_3014 + 1 + j_3014];
17        int i_3014 = 0, j_3014 = 0;
18        // Initial index of merged subarray array
19        int k_3014 = l_3014;
20        while (i_3014 < n1_3014 && j_3014 < n2_3014) {
21            if (L_3014[i_3014] <= R_3014[j_3014]) {
22                arr_3014[k_3014] = L_3014[i_3014];
23                i_3014++;
24            } else {
25                arr_3014[k_3014] = R_3014[j_3014];
26                j_3014++;
27            }
28            k_3014++;
29        }
30    }
31 }
```

```

31      /* Copy remaining elements of L[] if any */
32      while (i_3014 < n1_3014) {
33          arr_3014[k_3014] = L_3014[i_3014];
34          i_3014++;
35          k_3014++;
36      }
37
38      /* Copy remaining elements of R[] if any */
39      while (j_3014 < n2_3014) {
40          arr_3014[k_3014] = R_3014[j_3014];
41          j_3014++;
42          k_3014++;
43      }
44  }
45  void sort_3014(int arr_3014[], int l_3014, int r_3014) {
46      if (l_3014 < r_3014) {
47          // Find the middle point
48          int m_3014 = (l_3014 + r_3014) / 2;
49          // Sort first and second halves
50          sort_3014(arr_3014, l_3014, m_3014);
51          sort_3014(arr_3014, m_3014 + 1, r_3014);
52          // Merge the sorted halves
53          merge_3014(arr_3014, l_3014, m_3014, r_3014);
54      }
55  }
56
57      /* A utility function to print array of size n */
58      static void printArray_3014(int arr_3014[]) {
59          int n_3014 = arr_3014.length;
60          for (int i_3014 = 0; i_3014 < n_3014; ++i_3014)
61              System.out.print(arr_3014[i_3014] + " ");
62          System.out.println();
63      }
64
65      public static void main(String args[]) {
66          int arr_3014[] = {12, 11, 13, 5, 6, 7};
67          System.out.println("Sebelum terurut");
68          printArray_3014(arr_3014);
69          MergeSort_2511533014 ob_3014 = new MergeSort_2511533014();
70          ob_3014.sort_3014(arr_3014, 0, arr_3014.length - 1);
71          System.out.println("\nSesudah Terurut menggunakan merge Sort");
72          printArray_3014(arr_3014);
73      }
74  }

```

Gambar 2.5 Kode program praktikum class “MergeSort”

Program MergeSort_2511533014 digunakan untuk mengurutkan data dalam array menggunakan algoritma Merge Sort. Method sort_3014() membagi array menjadi beberapa bagian kecil secara rekursif hingga hanya tersisa satu elemen, kemudian method merge_3014() menggabungkan kembali bagian-bagian tersebut dalam urutan yang benar. Array sementara L_3014 dan R_3014 digunakan untuk menyimpan data yang akan dibandingkan saat proses penggabungan. Setelah seluruh proses selesai, data yang awalnya belum terurut berhasil diurutkan secara ascending (dari kecil ke besar) dan ditampilkan menggunakan method printArray_3014().

Berikut penjelasan langkah-langkah kerjanya:

1. void merge_3014(int arr_3014[], int l_3014, int m_3014, int r_3014)

→ Method untuk menggabungkan dua subarray yang sudah terurut menjadi satu array terurut.

2. `int n1_3014 = m_3014 - l_3014 + 1;`

→ Menghitung jumlah elemen pada subarray kiri.

3. `int n2_3014 = r_3014 - m_3014;`

→ Menghitung jumlah elemen pada subarray kanan.

4. `int L_3014[] = new int[n1_3014];`

→ Membuat array sementara untuk bagian kiri.

5. `int R_3014[] = new int[n2_3014];`

→ Membuat array sementara untuk bagian kanan.

6. `L_3014[i_3014] = arr_3014[l_3014 + i_3014];`

→ Menyalin data dari array utama ke array kiri.

7. `R_3014[j_3014] = arr_3014[m_3014 + 1 + j_3014];`

→ Menyalin data dari array utama ke array kanan.

`int i_3014 = 0, j_3014 = 0;`

→ Menyiapkan indeks awal untuk array kiri dan kanan.

8. `int k_3014 = l_3014;`

→ Menentukan posisi awal penggabungan pada array utama.

9. `while (i_3014 < n1_3014 && j_3014 < n2_3014)`

→ Membandingkan elemen array kiri dan kanan selama masih ada data.

10. `if (L_3014[i_3014] <= R_3014[j_3014])`

→ Memilih elemen yang lebih kecil untuk dimasukkan ke array utama.

11. `arr_3014[k_3014] = L_3014[i_3014];`

→ Memasukkan elemen dari array kiri ke array utama.

12. `arr_3014[k_3014] = R_3014[j_3014];`

- Memasukkan elemen dari array kanan ke array utama.
13. `k_3014++;`
- Berpindah ke posisi berikutnya pada array utama.
14. `while (i_3014 < n1_3014)`
- Menyalin sisa elemen array kiri jika masih ada.
15. `while (j_3014 < n2_3014)`
- Menyalin sisa elemen array kanan jika masih ada.
16. `void sort_3014(int arr_3014[], int l_3014, int r_3014)`
- Method utama Merge Sort untuk membagi dan mengurutkan array.
17. `if (l_3014 < r_3014)`
- Memastikan array masih dapat dibagi menjadi beberapa bagian.
18. `int m_3014 = (l_3014 + r_3014) / 2;`
- Menentukan indeks tengah array.
19. `sort_3014(arr_3014, l_3014, m_3014);`
- Mengurutkan bagian kiri array secara rekursif.
20. `sort_3014(arr_3014, m_3014 + 1, r_3014);`
- Mengurutkan bagian kanan array secara rekursif.
21. `merge_3014(arr_3014, l_3014, m_3014, r_3014);`
- Menggabungkan kembali dua bagian array yang sudah terurut.
22. `static void printArray_3014(int arr_3014[])`
- Method untuk menampilkan isi array.
23. `for (int i_3014 = 0; i_3014 < n_3014; ++i_3014)`
- Melakukan perulangan untuk mengakses seluruh elemen array.
24. `System.out.print(arr_3014[i_3014] + " ");`

→ Menampilkan setiap elemen array.

25. `public static void main(String args[])`

→ Method utama yang pertama kali dijalankan.

26. `int arr_3014[] = {12, 11, 13, 5, 6, 7};`

→ Membuat data yang akan diurutkan.

27. `System.out.println("Sebelum terurut");`

→ Menampilkan keterangan sebelum proses pengurutan.

28. `printArray_3014(arr_3014);`

→ Menampilkan isi array sebelum diurutkan.

29. `MergeSort_2511533014 ob_3014 = new MergeSort_2511533014();`

→ Membuat objek dari kelas MergeSort.

30. `ob_3014.sort_3014(arr_3014, 0, arr_3014.length - 1);`

→ Memanggil method Merge Sort untuk mengurutkan seluruh data.

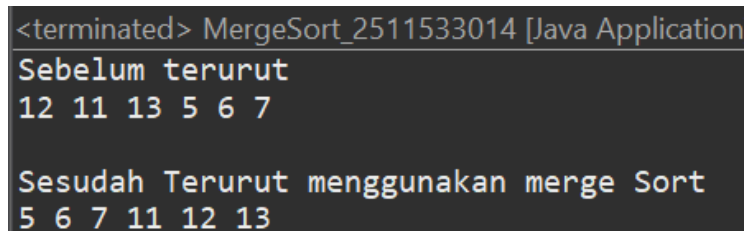
31. `System.out.println("\nSesudah Terurut menggunakan merge Sort");`

→ Menampilkan keterangan setelah proses pengurutan.

32. `printArray_3014(arr_3014);`

→ Menampilkan hasil array yang sudah terurut.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.6 dibawah ini.



```
<terminated> MergeSort_2511533014 [Java Application]
Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan merge Sort
5 6 7 11 12 13
```

Gambar 2.6 Output kode program class “MergeSort”

2.4.2 Analisis

Output menunjukkan bahwa sebelum proses Merge Sort, data dalam array masih acak yaitu 12, 11, 13, 5, 6, 7. Setelah method `sort_3014()` dijalankan, array dibagi menjadi beberapa bagian kecil, kemudian digabungkan kembali secara terurut melalui method `merge_3014()`. Hasil akhirnya adalah 5, 6, 7, 11, 12, 13, yang menunjukkan bahwa algoritma Merge Sort berhasil mengurutkan data secara ascending (dari kecil ke besar) dengan benar.

2.5 Praktikum Class “BubbleSortGUI”

```
1 package pekan8_2511533014;
2
3 import java.awt.EventQueue;
4
24
25 public class BubbleSortGUI_2511533014 extends JFrame {
26     private static final long serialVersionUID = 1L;
27
28     private int[] array_3014;
29     private JLabel[] labelArray_3014;
30     private JButton stepButton_3014, resetButton_3014, setButton_3014;
31     private JTextField inputField_3014;
32     private JPanel panelArray_3014;
33     private JTextArea stepArea_3014;
34
35     private int i_3014, j_3014;
36     private boolean sorting_3014 = false;
37     private int stepCount_3014 = 1;
38
39     public BubbleSortGUI_2511533014() {
40         setTitle("Bubble Sort: Langkah per Langkah");
41         setSize(750, 400);
42         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
43         setLocationRelativeTo(null);
44         setLayout(new BorderLayout());
45
46         // Panel input
47         JPanel inputPanel_3014 = new JPanel(new FlowLayout());
48         inputField_3014 = new JTextField(30);
49         setButton_3014 = new JButton("Set Array");
50
51         inputPanel_3014.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
52         inputPanel_3014.add(inputField_3014);
53         inputPanel_3014.add(setButton_3014);
```

```

55         // Panel array visual
56         panelArray_3014 = new JPanel(new FlowLayout());
57
58         // Panel kontrol
59         JPanel controlPanel_3014 = new JPanel();
60         stepButton_3014 = new JButton("Langkah Selanjutnya");
61         resetButton_3014 = new JButton("Reset");
62         stepButton_3014.setEnabled(false);
63
64         controlPanel_3014.add(stepButton_3014);
65         controlPanel_3014.add(resetButton_3014);
66
67         // Area teks log langkah
68         stepArea_3014 = new JTextArea(8, 40);
69         stepArea_3014.setEditable(false);
70         stepArea_3014.setFont(new Font("Monospaced", Font.PLAIN, 14));
71         JScrollPane scrollPane_3014 = new JScrollPane(stepArea_3014);
72
73         // Tambahkan panel ke frame
74         add(inputPanel_3014, BorderLayout.NORTH);
75         add(panelArray_3014, BorderLayout.CENTER);
76         add(controlPanel_3014, BorderLayout.SOUTH);
77         add(scrollPane_3014, BorderLayout.EAST);
78
79         // Event tombol
80         setButton_3014.addActionListener(e -> setArrayFromInput_3014());
81         stepButton_3014.addActionListener(e -> performStep_3014());
82         resetButton_3014.addActionListener(e -> reset_3014());
83     }

```

```

85     private void setArrayFromInput_3014() {
86         String text_3014 = inputField_3014.getText().trim();
87         if (text_3014.isEmpty()) return;
88         String[] parts_3014 = text_3014.split(",");
89         array_3014 = new int[parts_3014.length];
90         try {
91             for (int k_3014 = 0; k_3014 < parts_3014.length; k_3014++) {
92                 array_3014[k_3014] = Integer.parseInt(parts_3014[k_3014].trim());
93             }
94         } catch (NumberFormatException e) {
95             JOptionPane.showMessageDialog(this,
96                 "Masukkan hanya angka yang dipisahkan dengan koma",
97                 "Error",
98                 JOptionPane.ERROR_MESSAGE);
99             return;
100         }

```

```

101
102         i_3014 = 0;
103         j_3014 = 0;
104         stepCount_3014 = 1;
105         sorting_3014 = true;
106         stepButton_3014.setEnabled(true);
107         stepArea_3014.setText("");
108         panelArray_3014.removeAll();
109         labelArray_3014 = new JLabel[array_3014.length];
110         for (int k_3014 = 0; k_3014 < array_3014.length; k_3014++) {
111             labelArray_3014[k_3014] = new JLabel(String.valueOf(array_3014[k_3014]));
112             labelArray_3014[k_3014].setFont(new Font("Arial", Font.BOLD, 24));
113             labelArray_3014[k_3014].setOpaque(true);
114             labelArray_3014[k_3014].setBackground(Color.WHITE);
115             labelArray_3014[k_3014].setBorder(BorderFactory.createLineBorder(Color.BLACK));
116             labelArray_3014[k_3014].setPreferredSize(new Dimension(50, 50));
117             labelArray_3014[k_3014].setHorizontalAlignment(SwingConstants.CENTER);
118             panelArray_3014.add(labelArray_3014[k_3014]);
119         }
120
121         panelArray_3014.revalidate();
122         panelArray_3014.repaint();
123     }
124

```

```

125 private void performStep_3014() {
126     if (i_3014 < array_3014.length && sorting_3014) {
127         int key_3014 = array_3014[i_3014];
128         j_3014 = i_3014 - 1;
129
130         StringBuilder stepLog_3014 = new StringBuilder();
131         stepLog_3014.append("Langkah ").append(stepCount_3014)
132             .append(": Memasukkan ").append(key_3014).append("\n");
133         while (j_3014 >= 0 && array_3014[j_3014] > key_3014) {
134             array_3014[j_3014 + 1] = array_3014[j_3014];
135             j_3014--;
136         }
137         array_3014[j_3014 + 1] = key_3014;
138         updateLabels_3014();
139         stepLog_3014.append("Hasil: ").append(arrayToString_3014(array_3014)).append("\n\n");
140         stepArea_3014.append(stepLog_3014.toString());
141         i_3014++;
142         stepCount_3014++;
143     }
144     if (i_3014 == array_3014.length) {
145         sorting_3014 = false;
146         stepButton_3014.setEnabled(false);
147         JOptionPane.showMessageDialog(this, "Sorting selesai!");
148     }
149 }
150 private void updateLabels_3014() {
151     for (int k_3014 = 0; k_3014 < array_3014.length; k_3014++) {
152         labelArray_3014[k_3014].setText(String.valueOf(array_3014[k_3014]));
153     }
154 }
155 private void reset_3014() {
156     inputField_3014.setText("");
157     panelArray_3014.removeAll();
158     panelArray_3014.revalidate();
159     panelArray_3014.repaint();
160     stepArea_3014.setText("");
161     stepButton_3014.setEnabled(false);
162     sorting_3014 = false;
163     i_3014 = 0;
164     j_3014 = 0;
165     stepCount_3014 = 1;
166 }
167
168 private String arrayToString_3014(int[] arr_3014) {
169     StringBuilder sb_3014 = new StringBuilder();
170     for (int k_3014 = 0; k_3014 < arr_3014.length; k_3014++) {
171         sb_3014.append(arr_3014[k_3014]);
172         if (k_3014 < arr_3014.length - 1) sb_3014.append(", ");
173     }
174     return sb_3014.toString();
175 }
176
177 public static void main(String[] args) {
178     javax.swing.SwingUtilities.invokeLater(() -> {
179         BubleSortGUI_2511533014 gui_3014 = new BubleSortGUI_2511533014();
180         gui_3014.setVisible(true);
181     });
182 }

```

Gambar 2.7 Kode Program Praktikum class “BubleSortGUI”

Berikut penjelasan langkah-langkah kerjanya:

Variabel / Index Penting

1. int[] array_3014

→ Menyimpan data angka yang akan diurutkan.

2. JLabel[] labelArray_3014

→ Menampilkan setiap elemen array dalam bentuk visual pada GUI.

3. `int i_3014`

→ Indeks utama yang digunakan pada proses pengurutan.

4. `int j_3014`

→ Indeks untuk membandingkan dan menggeser elemen selama proses sorting.

5. `boolean sorting_3014`

→ Menandakan apakah proses pengurutan masih berjalan atau sudah selesai.

6. `int stepCount_3014`

→ Menghitung jumlah langkah pengurutan yang telah dilakukan.

Constructor `BubleSortGUI_2511533014()`

1. `setTitle("Buble Sort: Langkah per Langkah");`

→ Memberikan judul pada jendela GUI.

2. `setSize(750, 400);`

→ Mengatur ukuran jendela aplikasi.

3. `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);`

→ Menutup aplikasi saat tombol close ditekan.

4. `inputField_3014 = new JTextField(30);`

→ Membuat kotak input untuk memasukkan data.

5. `setButton_3014 = new JButton("Set Array");`

→ Membuat tombol untuk memasukkan array.

6. `stepButton_3014 = new JButton("Langkah Selanjutnya");`

→ Membuat tombol untuk menjalankan proses sorting per langkah.

7. `resetButton_3014 = new JButton("Reset");`

→ Membuat tombol untuk mengembalikan program ke kondisi awal.

8. `stepArea_3014 = new JTextArea(8, 40);`

→ Membuat area teks untuk menampilkan log proses sorting.

9. `add(...)`

→ Menambahkan seluruh komponen GUI ke dalam frame.

10. `addActionListener(...)`

→ Menghubungkan tombol dengan method yang akan dijalankan saat diklik.

Method `setArrayFromInput_3014()`

1. `String text_3014 = inputField_3014.getText().trim();`

→ Mengambil data dari kotak input.

2. `String[] parts_3014 = text_3014.split(",");`
→ Memisahkan angka berdasarkan tanda koma.
3. `array_3014 = new int[parts_3014.length];`
→ Membuat array sesuai jumlah data yang dimasukkan.
4. `Integer.parseInt(...)`
→ Mengubah data String menjadi integer.
5. `i_3014 = 0;`
→ Mengatur indeks awal sorting.
6. `sorting_3014 = true;`
→ Menandakan proses sorting siap dijalankan.
7. `stepButton_3014.setEnabled(true);`
→ Mengaktifkan tombol langkah selanjutnya.
8. `labelArray_3014 = new JLabel[array_3014.length];`
→ Membuat label visual untuk setiap elemen array.
9. `panelArray_3014.add(labelArray_3014[k_3014]);`
→ Menampilkan elemen array pada panel GUI.

Method performStep_3014()

1. `if (i_3014 < array_3014.length && sorting_3014)`
→ Memastikan proses sorting masih berlangsung.
2. `int key_3014 = array_3014[i_3014];`
→ Menyimpan elemen yang akan ditempatkan pada posisi yang benar.
3. `j_3014 = i_3014 - 1;`
→ Menentukan indeks pembandingan.
4. `while (j_3014 >= 0 && array_3014[j_3014] > key_3014)`
→ Membandingkan dan menggeser elemen yang lebih besar.
5. `array_3014[j_3014 + 1] = array_3014[j_3014];`
→ Menggeser elemen ke kanan.
6. `j_3014--;`
→ Berpindah ke elemen sebelumnya.
7. `array_3014[j_3014 + 1] = key_3014;`
→ Menempatkan elemen pada posisi yang benar.
8. `updateLabels_3014();`

→ Memperbarui tampilan array pada GUI.

9. `stepArea_3014.append(...)`

→ Menampilkan hasil langkah sorting pada area log.

10. `i_3014++;`

→ Berpindah ke langkah berikutnya.

11. `stepCount_3014++;`

→ Menambah nomor langkah.

12. `JOptionPane.showMessageDialog(...)`

→ Menampilkan pesan ketika sorting selesai.

Method `updateLabels_3014()`

1. `for (int k_3014 = 0; k_3014 < array_3014.length; k_3014++)`

→ Mengakses seluruh elemen array.

2. `labelArray_3014[k_3014].setText(...)`

→ Memperbarui nilai yang ditampilkan pada label GUI.

Method `reset_3014()`

1. `inputField_3014.setText("");`

→ Mengosongkan kotak input.

2. `panelArray_3014.removeAll();`

→ Menghapus seluruh tampilan array.

3. `stepArea_3014.setText("");`

→ Menghapus log langkah sorting.

4. `stepButton_3014.setEnabled(false);`

→ Menonaktifkan tombol langkah.

5. `sorting_3014 = false;`

→ Menghentikan proses sorting.

6. `i_3014 = 0;`

→ Mengembalikan indeks ke nilai awal.

7. `stepCount_3014 = 1;`

→ Mengatur ulang nomor langkah.

Method `arrayToString_3014()`

1. `StringBuilder sb_3014 = new StringBuilder();`

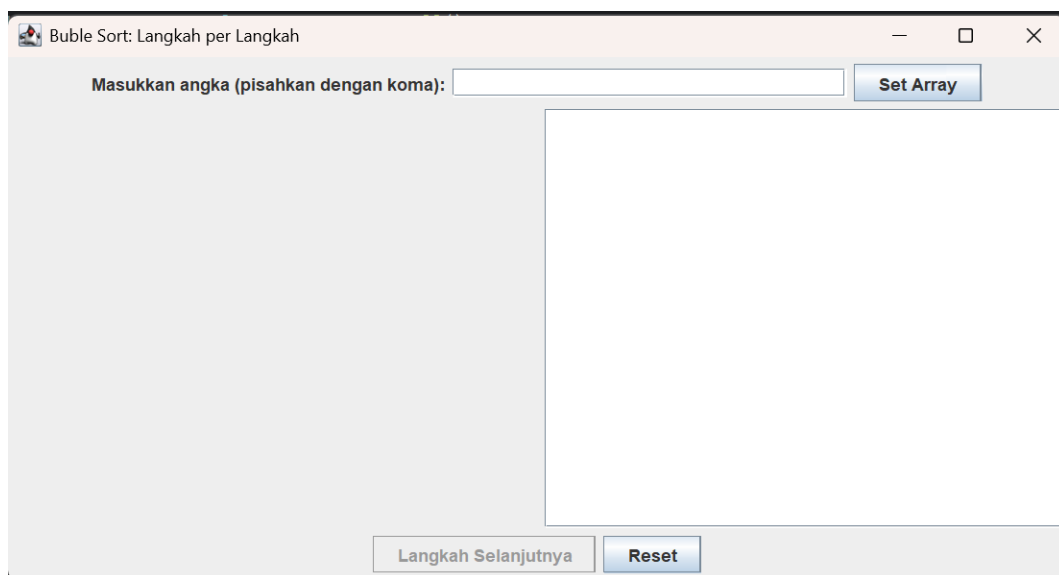
→ Membuat objek untuk menyusun teks array.

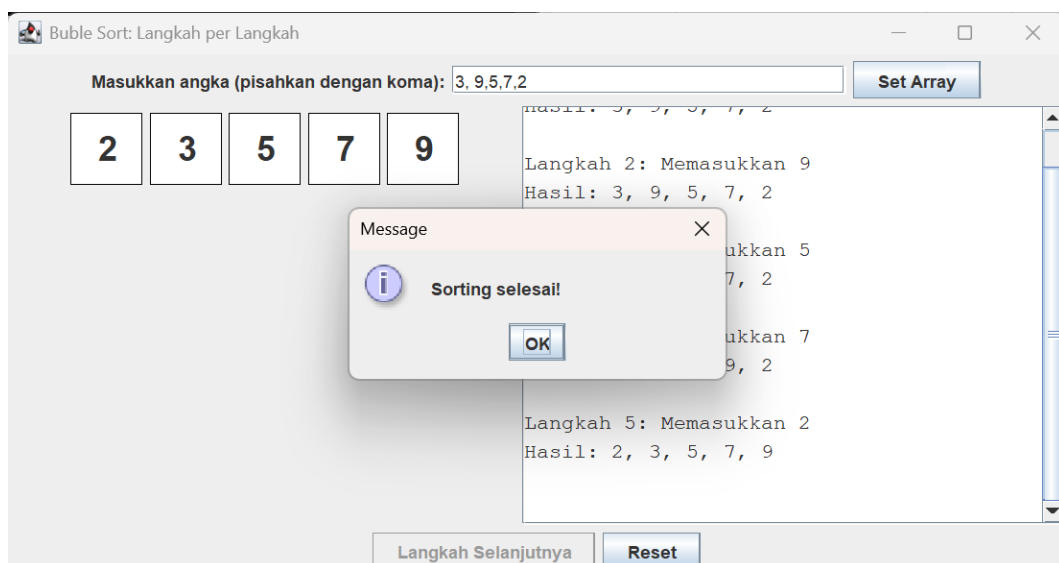
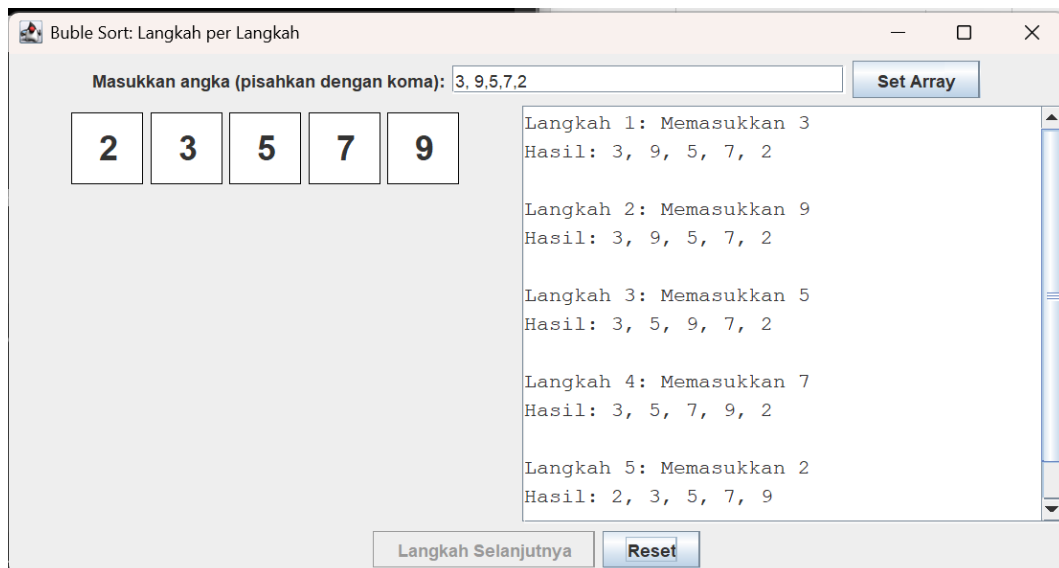
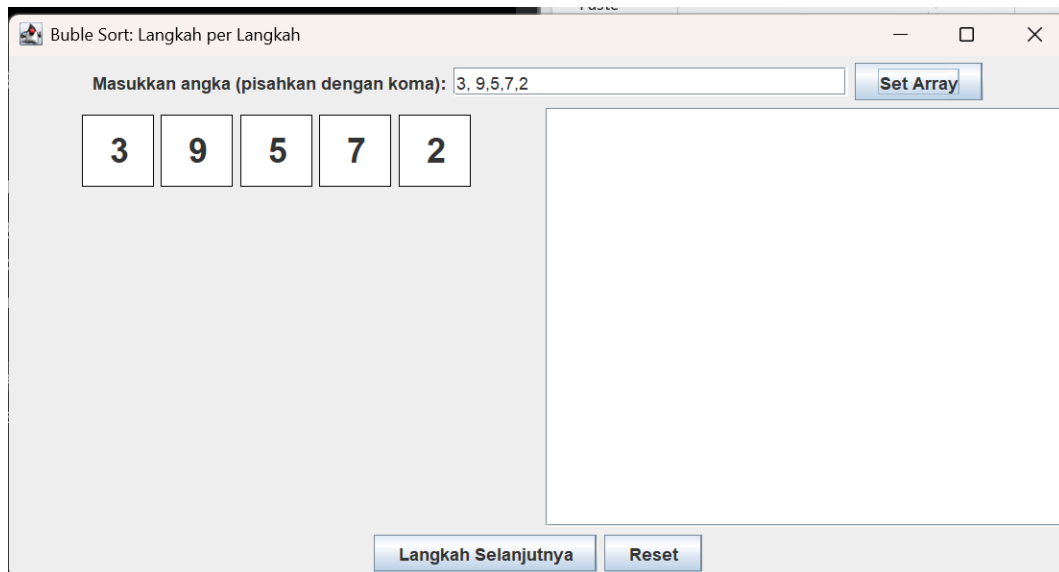
2. `sb_3014.append(arr_3014[k_3014]);`
→ Menambahkan elemen array ke dalam String.
3. `return sb_3014.toString();`
→ Mengembalikan array dalam bentuk teks.

Method `main()`

1. `SwingUtilities.invokeLater(...)`
→ Menjalankan GUI pada thread Swing.
2. `BubleSortGUI_2511533014 gui_3014 = new BubleSortGUI_2511533014();`
→ Membuat objek GUI.
3. `gui_3014.setVisible(true);`
→ Menampilkan jendela aplikasi ke layar.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.8 dibawah ini.





2.5.1 Analisis

1. Tampilan Awal Program

Saat program dijalankan, pengguna akan melihat kolom input untuk memasukkan angka yang dipisahkan dengan koma, tombol Set Array, tombol Langkah Selanjutnya (masih nonaktif), dan tombol Reset.

2. Memasukkan Data

- Pengguna memasukkan data, misalnya 3, 9, 5, 7, 2 lalu menekan tombol Set Array.
- Program memecah data berdasarkan tanda koma dan mengubahnya menjadi array integer.
- Data kemudian ditampilkan dalam bentuk kotak-kotak pada panel sebelah kiri.

3. Menjalankan Sorting per Langkah

- Setelah tombol Set Array ditekan, tombol Langkah Selanjutnya akan aktif.
- Setiap kali tombol Langkah Selanjutnya ditekan, program menjalankan satu langkah proses pengurutan.
- Program mengambil nilai `key_3014` kemudian membandingkannya dengan elemen sebelumnya.
- Jika ada elemen yang lebih besar dari `key_3014`, elemen tersebut akan digeser ke kanan.
- Setelah posisi yang sesuai ditemukan, `key_3014` ditempatkan pada posisi tersebut.
- Hasil setiap langkah ditampilkan pada area log di sebelah kanan.

4. Menampilkan Hasil

- Setelah seluruh elemen diproses, tombol Langkah Selanjutnya akan dinonaktifkan.
- Program menampilkan pesan "Sorting selesai!" menggunakan JOptionPane.

5. Reset Program

Jika tombol Reset ditekan, seluruh data input, tampilan array, dan log langkah akan dihapus sehingga pengguna dapat memasukkan data baru.

Sebelum sorting, data berada dalam kondisi belum terurut yaitu **3, 9, 5, 7, 2**. Selama proses pengurutan, program membandingkan setiap elemen dengan elemen sebelumnya dan menggeser elemen yang lebih besar ke kanan hingga ditemukan posisi yang sesuai. Setelah seluruh langkah selesai, data berhasil diurutkan secara ascending (kecil ke besar) menjadi **2, 3, 5, 7, 9**. Hasil ini menunjukkan bahwa algoritma yang digunakan bekerja dengan baik untuk mengurutkan data yang belum terurut.

2.6 Laporan Tugas Praktikum

2.6.1 Tugas Class “Lagu_2511533014”

```

1 package pekan8_2511533014;
2
3 public class Lagu_2511533014 {
4     String judul_3014;
5     String penyanyi_3014;
6     int durasi_3014;
7
8     public Lagu_2511533014(String judul_3014, String penyanyi_3014, int durasi_3014) {
9         this.judul_3014 = judul_3014;
10        this.penyanyi_3014 = penyanyi_3014;
11        this.durasi_3014 = durasi_3014;
12    }
13 }

```

2.6.2 Tugas Class “Sorting_2511533014”

```

1 package pekan8_2511533014;
2 import java.util.Scanner;
3 public class Sorting_2511533014 {
4     Lagu_2511533014[] dataLagu_3014 = new Lagu_2511533014[20];
5     int jumlahData_3014 = 3;
6     void inputData_3014() {
7         dataLagu_3014[0] = new Lagu_2511533014(
8             "Mio Cristo Piange Diamanti",
9             "Caparezza",
10            270);
11         dataLagu_3014[1] = new Lagu_2511533014(
12             "La Rumba Del Perdon",
13             "Juanes",
14            252);
15         dataLagu_3014[2] = new Lagu_2511533014(
16             "La Perla",
17             "Calle 13",
18            196);
19     }
20
21     int partition_3014(Lagu_2511533014 arr_3014[], int low_3014, int high_3014) {
22         int pivot_3014 = arr_3014[high_3014].durasi_3014;
23         int i_3014 = low_3014 - 1;
24         for (int j_3014 = low_3014; j_3014 < high_3014; j_3014++) {
25             if (arr_3014[j_3014].durasi_3014 < pivot_3014) {
26                 i_3014++;
27                 Lagu_2511533014 temp_3014 = arr_3014[i_3014];
28                 arr_3014[i_3014] = arr_3014[j_3014];
29                 arr_3014[j_3014] = temp_3014;
30             }
31         }
32
33         Lagu_2511533014 temp_3014 = arr_3014[i_3014 + 1];
34         arr_3014[i_3014 + 1] = arr_3014[high_3014];
35         arr_3014[high_3014] = temp_3014;
36
37         return i_3014 + 1;
38     }
39
40     void quickSort_3014(Lagu_2511533014 arr_3014[],
41         int low_3014,
42         int high_3014) {
43         if (low_3014 < high_3014) {
44             int pi_3014 = partition_3014(arr_3014,
45                 low_3014,
46                 high_3014);
47             quickSort_3014(arr_3014,
48                 low_3014,
49                 pi_3014 - 1);
50             quickSort_3014(arr_3014,
51                 pi_3014 + 1,
52                 high_3014);
53         }
54     }
55
56     void tampilData_3014() {
57         for (int i_3014 = 0; i_3014 < jumlahData_3014; i_3014++) {
58             System.out.println((i_3014 + 1) + ". "
59                 + dataLagu_3014[i_3014].judul_3014
60                 + " - "
61                 + dataLagu_3014[i_3014].durasi_3014
62                 + " detik");
63         }
64     }

```

```

66 public static void main(String[] args) {
67     Scanner input_3014 = new Scanner(System.in);
68     Sorting_2511533014 data_3014 =
69         new Sorting_2511533014();
70     data_3014.inputData_3014();
71     System.out.println(
72         "=== Sorting Playlist NIM: 2511533014 ===");
73     System.out.print(
74         "Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): ");
75     int pilihan_3014 = input_3014.nextInt();
76     System.out.println("\nData Sebelum Sorting:");
77     data_3014.tampilData_3014();
78     if (pilihan_3014 == 2) {
79         data_3014.quickSort_3014(
80             data_3014.dataLagu_3014,
81             0,
82             data_3014.jumlahData_3014 - 1);
83         System.out.println(
84             "\nData Setelah Quick Sort (Durasi Asc):");
85         data_3014.tampilData_3014();
86     } else {
87         System.out.println(
88             "\nAlgoritma belum diimplementasikan.");
89     }
90
91     input_3014.close();
92 }
93 }

```

Program `Sorting_2511533014` digunakan untuk mengurutkan data playlist lagu yang disimpan dalam array menggunakan algoritma Quick Sort berdasarkan durasi lagu dari yang terpendek hingga terpanjang. Class `Lagu_2511533014` berfungsi menyimpan data lagu berupa judul, penyanyi, dan durasi, sedangkan class `Sorting_2511533014` berisi data lagu, method `inputData_3014()` untuk mengisi data, `quickSort_3014()` untuk melakukan pengurutan, dan `tampilData_3014()` untuk menampilkan data sebelum dan sesudah sorting. Proses pengurutan dilakukan dengan membandingkan durasi lagu menggunakan metode partition hingga seluruh data tersusun secara ascending. Program kemudian menampilkan hasil pengurutan sesuai pilihan algoritma yang dipilih pengguna.

2.6.3 Output Program

```
<terminated> Sorting_2511533014 [Java Application] C:\Users\rara
=== Sorting Playlist NIM: 2511533014 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

Data Sebelum Sorting:
1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik

Data Setelah Quick Sort (Durasi Asc):
1. La Perla - 196 detik
2. La Rumba Del Perdon - 252 detik
3. Mio Cristo Piange Diamanti - 270 detik
```

2.6.4 Alasan memilih algoritma quick sort

Algoritma Quick Sort dipilih karena memiliki performa yang cepat dalam mengurutkan data dibandingkan beberapa algoritma sorting sederhana lainnya. Quick Sort bekerja dengan memilih sebuah pivot kemudian membagi data menjadi dua bagian, yaitu data yang lebih kecil dan lebih besar dari pivot. Rata-rata kompleksitas waktu Quick Sort adalah $O(n \log n)$, sehingga cukup efisien untuk mengurutkan data playlist lagu berdasarkan durasi. Selain itu, implementasi Quick Sort relatif mudah diterapkan pada array objek lagu dengan membandingkan atribut durasi. Oleh karena itu, Quick Sort cocok digunakan untuk mengurutkan data lagu dari durasi terpendek ke terpanjang.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting digunakan untuk mengurutkan data agar lebih terstruktur dan mudah diolah. Pada praktikum ini telah dipelajari dan diimplementasikan beberapa algoritma sorting, yaitu Shell Sort, Quick Sort, dan Merge Sort menggunakan bahasa pemrograman Java. Hasil pengujian menunjukkan bahwa setiap algoritma mampu mengurutkan data dengan benar meskipun memiliki cara kerja yang berbeda.

Selain itu, penerapan algoritma Quick Sort pada studi kasus playlist lagu berhasil mengurutkan data berdasarkan durasi dari yang terpendek hingga terpanjang. Melalui praktikum ini, pemahaman mengenai konsep pengurutan data, implementasi algoritma sorting, serta analisis hasil pengurutan dapat meningkat. Dengan demikian, tujuan praktikum telah tercapai dan memberikan pengalaman dalam menerapkan algoritma sorting pada permasalahan nyata.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, “Quick Sort Algorithm,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/quick-sort/>. [Accessed: 4-Jun-2026].
- [2] GeeksforGeeks, “Merge Sort Algorithm,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/merge-sort/>. [Accessed: 4-Jun-2026].
- [3] GeeksforGeeks, “Shell Sort,” GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/shell-sort/>. [Accessed: 4-Jun-2026].
- [4] Oracle Corporation, “The Java™ Tutorials,” Oracle. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>. [Accessed: 4-Jun-2026].